

ASSIGNMENT#2

NAME : MASHAAL ISHTIAQ

ROLL# : 22P-9326

SECTION : BCS-4C

```
#include <iostream>
#include <string>

using namespace std;

class Node
{
public:
string word;
int frequency;
Node *left;
Node *right;
int height;

Node(string word, int frequency) // parameterized constructor
{
this->word = word;
this->frequency = frequency;
this->left = nullptr;
this->right = nullptr;
this->height = 1;
}
};

class PrintWord
{
public:
static int count;
static string arr[20][2];

void addWord(string word, int frequency)
{
if (count < 20)
{
arr[count][0] = word;
arr[count][1] = to_string(frequency); // Convert frequency to string
count++;
sort();
}
else
{
cout << "Array is full, cannot add more words." << endl;
}
}

// void sort()
// {
// for (int i = 0; i < count - 1; ++i)
```

```
// {
// for (int j = i + 1; j < count; ++j)
// {
// if (stoi(arr[i][1]) < stoi(arr[j][1]))
// {
// swap(arr[i][0], arr[j][0]);
// swap(arr[i][1], arr[j][1]);
// }
// else if (stoi(arr[i][1]) == stoi(arr[j][1]) && arr[i][0] > arr[j][0])
// {
// swap(arr[i][0], arr[j][0]);
// swap(arr[i][1], arr[j][1]);
// }
// }
// }
// }

void sort()
{
for (int i = 0; i < count - 1; ++i)
{
for (int j = i + 1; j < count; ++j)
{
if (stoi(arr[i][1]) < stoi(arr[j][1]) || (stoi(arr[i][1]) == stoi(arr[j][1]) && arr[i][0] > arr[j][0]))
{
swap(arr[i][0], arr[j][0]);
swap(arr[i][1], arr[j][1]);
}
}
}
}

void display()
{
cout << "Word\tFrequency" << endl;
for (int i = 0; i < count; ++i)
{
cout << arr[i][0] << "\t" << arr[i][1] << endl;
}
count = 0; // Reset count after displaying
}
// friend AVL;
};

class AVL
{
public:
Node *root;

AVL()
{
root = nullptr;
```

```
}

Node *insert(Node *node, string word, int frequency)
{
if (node == NULL)
{
Node *temp = new Node(word, frequency);
if (root == NULL)
{
root = temp;
}

return temp; // return new Node(word, frequency);
}

if (word < node->word)
{
node->left = insert(node->left, word, frequency);
}
else if (word > node->word)
{
node->right = insert(node->right, word, frequency);
}
else
{
node->frequency += frequency; // word is already present therefore the frequency of the word is increment
}

node->height = updateHeight(node);

int balance = getBalance(node);

//____checking the balance factors

if (balance > 1 && word < node->left->word) // if the balance is greater than 1 -> it means that the left subtree is heavier than the right subtree and we will in insert in left subtree
{
return rightRotate(node);
}

else if (balance < -1 && word > node->right->word) // it checkss that if the word to be inserted is greater than the word of the right child of the node ->then the new node should be inserted into the right subtree
{
return leftRotate(node);
}

else if (balance > 1 && word > node->left->word) // as the word is greater than the left child so it should be inserted in the left subtree but as -> left subtree of the current node is too heavy than the right subtree -> so we need to perform double rotation so we can also balances the left sub tree
{
```

```
node->left = leftRotate(node->left);
return rightRotate(node);
}

else if (balance < -1 && word < node->right->word)
{
node->right = rightRotate(node->right);
return leftRotate(node);
}

return node;
}

//_____balance

int getBalance(Node *node)
{
if (node == NULL)
{
return 0;
}
return getHeight(node->left) - getHeight(node->right);
}

//___height

int getHeight(Node *node)
{
if (node == NULL)
return 0;

return node->height;
}

int updateHeight(Node *node)
{
if (node == NULL)
return 0;

int rightH = getHeight(node->right);
int leftH = getHeight(node->left);

return 1 + max(leftH, rightH); // return 1+[(left > right) ? left : right ]
}

//____rotations

Node *rightRotate(Node *a)
{
Node *temp = a->left; // left child of node -> a
temp->right = a; // right child of temp
```

```
a->left = NULL;

// updating the heights after rotating
a->height = updateHeight(a);
temp->height = updateHeight(temp);
root = temp;

return temp; // returning the new root
}

Node *leftRotate(Node *b)
{
Node *temp = b->right; // right child of node -> b
temp->left = b; // left child of temp

b->left = NULL;

// updating the heights after rotating
b->height = updateHeight(b);
temp->height = updateHeight(temp);
root = temp;

return temp; // returning the new root
}

void autoComplete(Node *node, string prefix, PrintWord &print_word)
{
if (node == nullptr)
return;

if (node->word.find(prefix) == 0)
{
// PrintWord print_word;
print_word.addWord(node->word, node->frequency);
}

// if (prefix <= node->word)
autoComplete(node->left, prefix, print_word);

// if (prefix < node->word)
autoComplete(node->right, prefix, print_word);
}

// friend Print;
// friend int main();
};

int PrintWord::count = 0;
string PrintWord::arr[20][2];
int main()
{
AVL tree;
```

```
tree.root = NULL;

tree.insert(tree.root, "car", 10);
tree.insert(tree.root, "cat", 5);
tree.insert(tree.root, "calendar", 16);
tree.insert(tree.root, "cart", 3);
tree.insert(tree.root, "sit", 2);

string prefix;
cout << "Enter a prefix for auto-complete: ";
cin >> prefix;

PrintWord print_word; // Create an instance of Print
tree.autoComplete(tree.root, prefix, print_word);

// Calling addWord, sort, and display through the print_word instance(obj)
// print_word.addWord(tree.root->word, tree.root->frequency);

print_word.sort();
print_word.display();

return 0;
}
```

